



TECHNIKUM

WIEN

Threads (linux)

Threads introduction

- **a process:**
 - Bundles resources that belong together
 - Unit of resource ownership
 - virtual address space stores process image
 - has a defined execution order of instructions (an execution thread)
- **disadvantages**
 - 1 parallel instruction inside 1 process (single threaded)
 - complex communication between multiple processes (no shared address space with fork())

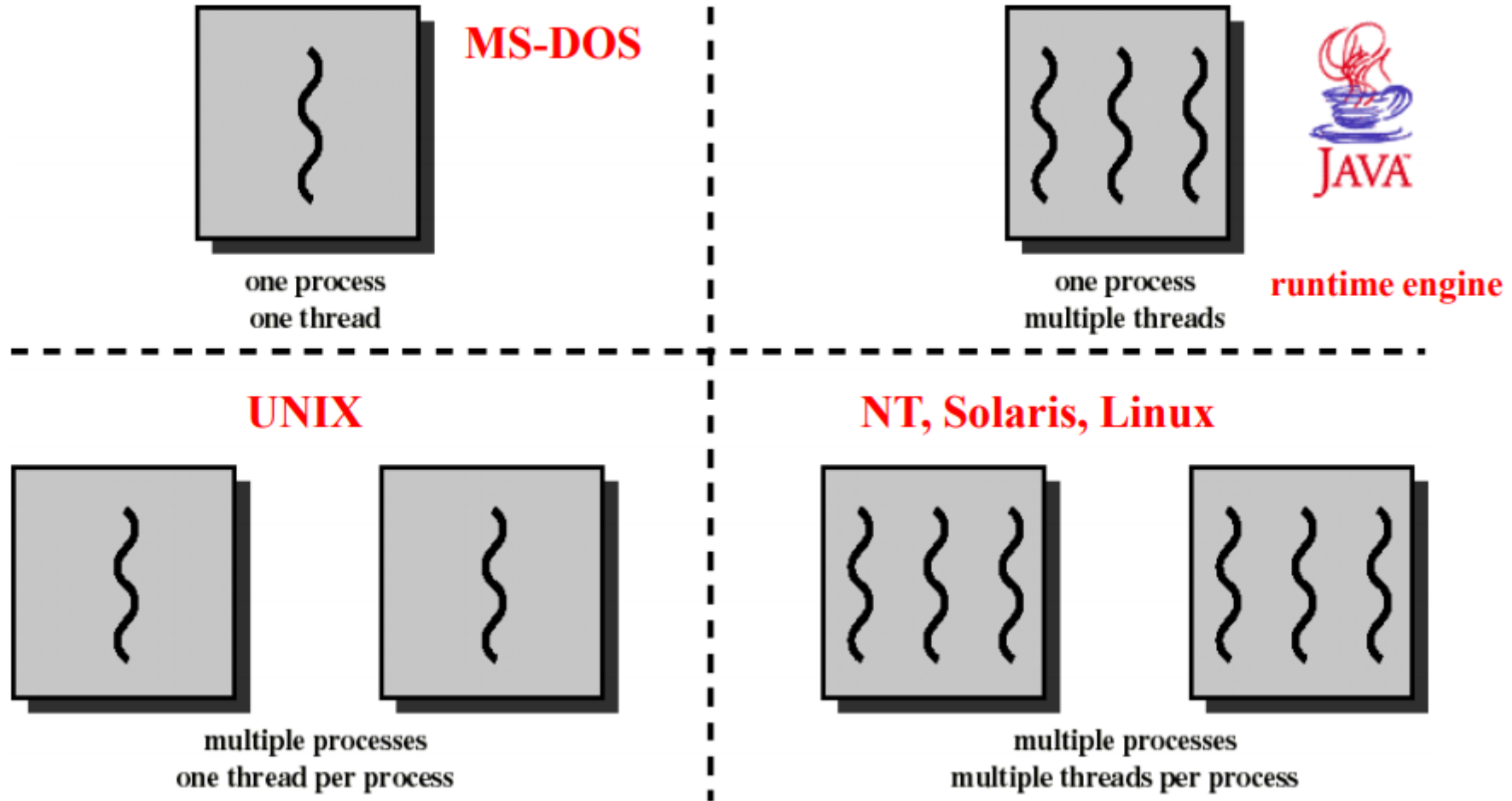
Threads

- **Threads overcome this problem by introducing**
 - **concurrent execution of instructions (multiple execution threads)**
 - **Shared address space**
 - **„lightweight process“**
 - **dispatchable unit of work**

Threads on OS level

- **Single threading**
 - Operating system is not aware of threads
- **Multi threading**
 - Operating system supports the execution of multiple threads in a process.

OS capabilities



Thread attributes

- **own execution state**
 - running, ready, blocked
 - (suspended on process level)
- **own (light weight) thread context**
- **own execution stack and memory**

- **shared address space (all threads of process)**
- **shared resources (e.g.: file handles)**

- **in case a process terminates, all related threads terminate as well**

Thread advantages over forked processes

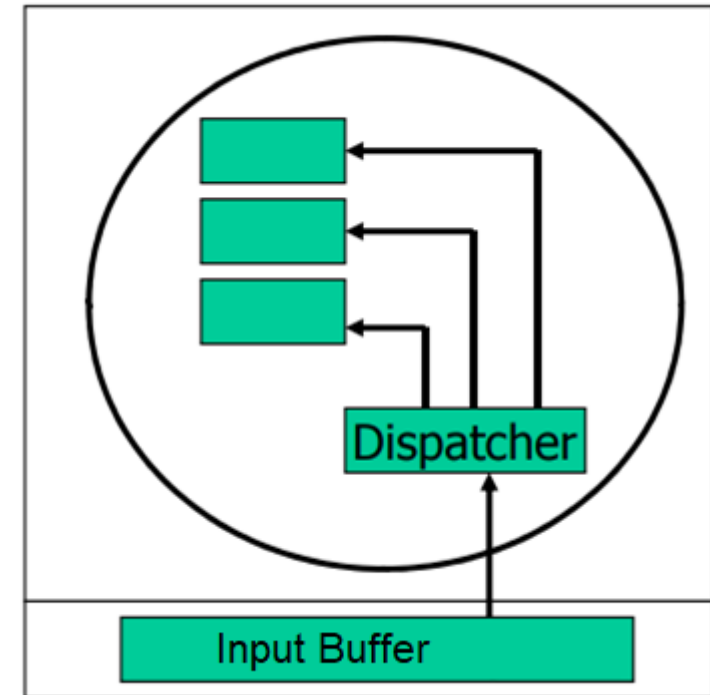
- less meta information (compared to fork)
- faster creation (compared to fork)
- faster termination (compared to fork)
- faster switch between threads
(compared to full context switch of processes)
- parallelism to gain performance (multi processor machines)
- Communication without kernel functions
 - Synchronization of critical sections necessary

Examples

- **word processing**
 - Auto Save
 - Spell Check
 - Formatting
 - Input / Output
- **webserver**
 - 1 thread per client request

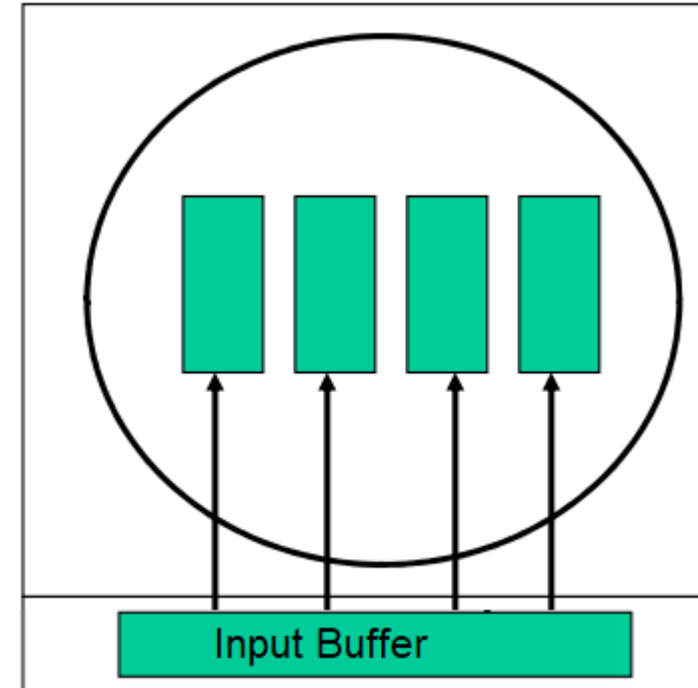
Threading – Dispatcher/Worker Model

- 1 thread receives the tasks and distributes it to the worker threads



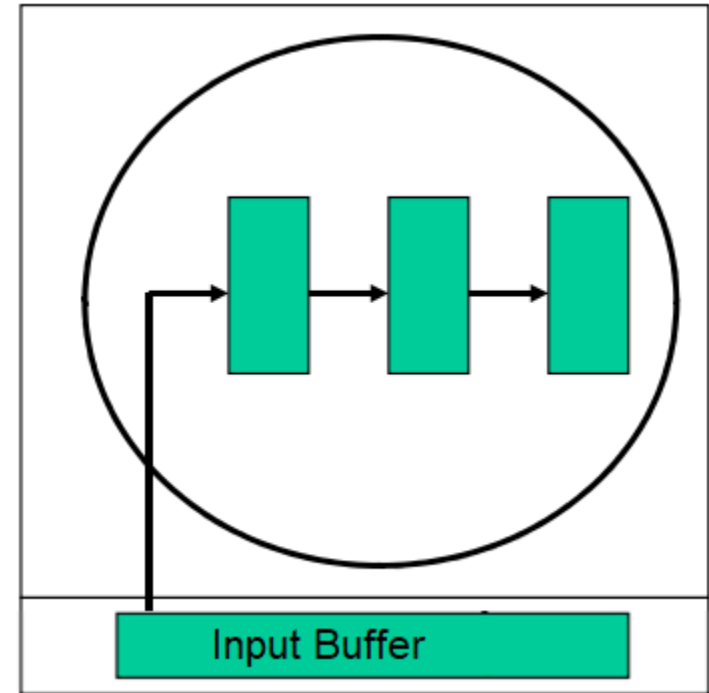
Threading – Team Model

- all threads are equal
- access to input buffer must be synchronized



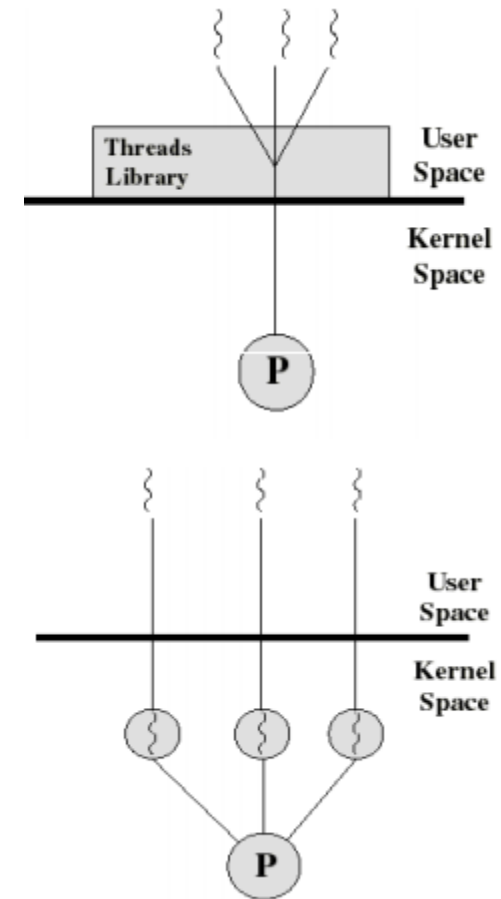
Threading – Pipeline Model

- Threads are logically organized as a chain of workers
- Each worker is working only on a part of the task



Kinds of threads

- **User level threads**
 - OS does not know about threads
 - Threading implemented at application layer
- **Kernel level threads**
 - Thread management supported by Operating system
 - APIs provided



Thread Implementation

- **pthread – library (POSIX Thread) defines a standard interface of around 100 procedures prefixed with pthread_ (pthread.h)**
- **pthread_create creates a thread based on passed in callback**
 - **when function behind the callback is done the thread terminates**
- **Linker flag -lpthread required**

example

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <pthread.h>
4
5 int finished=0;
6 void *test_thread(void *arg) {
7     int i;
8     printf("thread started\n");
9     pthread_detach(pthread_self());
10    for (i = 0; i < 100000000; i++); // long calculation...
11    finished=1;
12    printf("thread terminates ...\n");
13    return NULL;
14 }
15
16 int main(int argc, char *argv[]) {
17     int status;
18     pthread_t thread;
19     status = pthread_create(&thread, NULL, test_thread, NULL);
20     while (!finished) sleep (1);
21     printf("... end finished!\n");
22     return EXIT_SUCCESS;
23 }
```